



Programación con Objetos II

1037-2-G14

Grupo 15

Lucas Saenz

lsaenz05@outlook.com

1/07/2026

Índice

I. Catalogo (*Pag 3*)

II. Pedido (*Pag 3 y 4*)

III. Envío (*Pag 4 y 5*)

IV. Pago (*Pag 5*)

V. Notificaciones (*Pag 5 y 6*)

VI. Buscador (*Pag 6 y 7*)

VII. Reportes (*Pag 7*)

VIII. Test (*Pag 7*)

❖ 2.1 Catalogo

1. Decisiones de diseño

Se opto por emplear el patrón de diseño **Composite** ya que ofrece un manejo de la recursividad justo para este caso, permitiendo así que un **Paquete** pueda contener tanto **Productos** como otros **Paquetes** y tratarlos de la misma manera.

2. Detalles de implementación

Los **Item** cuentan con sus atributos base pero a su vez también permite agregar nuevos atributos dinámicamente usando un Map.

Como **Item** se entiende un **Producto** o un grupo de estos mismos, **Paquete**. Cada tipo de **Item** sabe cómo validarse validateItem, si pertenece a una categoría isCategory y calcula su precio final getFinalPrice de la manera que le corresponda.

3. Roles del patrón Composite

Component: **Item**

Leaf: **Producto**

Composite: **Paquete**

Client: **Pedido**

❖ 2.2 Pedido

1. Decisiones de diseño

Se opto por emplear el patrón de diseño **State** ya que permite delegar tareas sin necesidad usar condiciones en exceso, ofreciendo diferentes comportamientos dependiendo del estado.

2. Detalles de implementación

Según el **Estado** en que se encuentra el **Pedido** se pueden realizar ciertas acciones como agregar addItem o remover removeItem Ítems del carrito.

Cada **Estado** sabe a cuál se tiene que actualizar newState cuando avanzan las etapas del **Pedido** y si puede ser cancelado cancelled según la instancia en la que se encuentre.

Al cancelarse un **Pedido** que está en el proceso de preparación o ya fue enviado se genera y registra register una **NotaDeCredito** al cliente como reembolso de la compra.

3. Roles del patrón State

Context: **Pedido**

State: **Estado**

ConcreteState: **Borrador, Confirmado, EnPreparacion, Enviado, Entregado, Cancelado**

❖ 2.3 Envío

1. Decisiones de diseño

Se optó por emplear el patrón de diseño **Strategy** ya que permite ejecutar una misma acción de diferentes maneras según el método de envío que tenga seleccionado el **Pedido**.

2. Detalles de implementación

Un **Envío** permite calcular el costo para sus diferentes métodos calculateCost, dependiendo algunos de la **Dirección** y el peso o el precio. Cada método de **Envío** implementa una interface CorreoArgentina / EnvioExpress la cual se encarga de realizar el cálculo del costo según los parámetros enviados.

También se puede conocer cuánto tiempo va a tardar en llegar el **Producto** a destino shippingTime, algunos métodos dependen del stock disponible en la **Sucursal**.

3. Roles del patrón Strategy

Context: **Pedido**

Strategy: **Envío**

ConcreteStrategy: **Estándar, Express, Presencial**

❖ **2.4 Pago**

1. Decisiones de diseño

Se optó por emplear el patrón de diseño **Template Method** ya que ofrece la posibilidad de ejecutar un procedimiento pero con variaciones en sus etapas según cada método.

2. Detalles de implementación

Cada **Pago** a la hora de realizar una compra lleva a cabo un proceso processPayment que tiene siempre los mismos pasos pero con variaciones dependiendo el método de pago.

Se utiliza una **API** externa TarjetaAPI / TransferenciaAPI / BilleteraVirtualAPI correspondiente a cada método para realizar las comprobaciones necesarias de cada paso.

Según el método de pago seleccionado al finalizar la compra se genera y registra register un **CuponDePago**, un **ComprobanteTransferencia** o nada.

3. Roles del patrón Template Method

AbstractClass: **Pago**

ConcreteClass: **Tarjeta, BilleteraVirtual, Transferencia**

❖ **2.5 Notificaciones**

1. Decisiones de diseño

Se optó por emplear el patrón de diseño **Observer** considerando que cumple con la necesidad de conocer cuando sucede un cambio de estado en el **Pedido** y realizar acciones cuando esto suceda.

2. Detalles de implementación

Existen diversos tipos de **Notificación**, avisos por correo electrónico, generación de factura con comprobante fiscal y cupones de descuento por cancelación.

Un **Pedido** puede implementar varias notificaciones, cuando este sufre un cambio de **Estado** envía las notificaciones shotout correspondientes al cliente.

3. Roles del patrón Observer

Subject: **Pedido**

Observer: **Notificación**

ConcreteSubject: **Pedido**

ConcreteObserver: **Email, Factura, Fidelización**

❖ 2.6 Buscador

1. Decisiones de diseño

Se optó por emplear el patrón de diseño **Composite** ya que gracias a su recursividad permite juntar varios criterios de búsqueda tanto simples como complejos para hacer una única búsqueda.

2. Detalles de implementación

Un **Criterio** realiza un filtro filter sobre el catálogo y devuelve el mismo pero únicamente con los ítems que cumplen la condición de búsqueda.

Existen criterios simples como lo son por nombre, precio, categoría y disponibilidad.

Así como también están los criterios complejos que se conforman por AND, OR y NOT, estos permiten agrupar varios criterios tanto simples como complejos.

3. Roles del patrón Composite

Component: **Criterio**

Leaf: **Nombre, Precio, Categoría, Disponibilidad**

Composite: **AND, OR, NOT**

Client: (*) **Buscador**

❖ 2.8 Reportes

1. Decisiones de diseño

Se optó por emplear el patrón de diseño **Visitor** ya que ofrece la flexibilidad de poder agregar diferentes tipos de archivo a futuro sin necesidad de realizar modificaciones sobre las clases que lo utilizan.

2. Detalles de implementación

Para exportar un reporte, **Sucursal** cuenta con su método newReport el cual funciona como *accept* del *visit* que en este caso sería generateReport por el lado de **Reporte**.

La **Sucursal** se encarga de la lógica de negocio registrando cada **Venta** generada y organizando el catálogo según los ítems más vendidos en cierto periodo getBestSeller y delegando la exportación de esos datos al **Reporte**, el cual tiene diferentes formatos y permite que a futuro se implementen más.

3. Roles del patrón Visitor

Visitor: **Reporte**

ConcreteVisitor: **TXT, HTML, CSV**

Element: **Sucursal**

ConcreteElement: **Sucursal**

ObjectStructure: **Sucursal**

❖ Tests

Se cuenta con un **96,2%** de cobertura sobre todo el desarrollo, probando el funcionamiento tanto en conjunto como individualmente de cada componente.